

一、项目概述

Harmonize3D 是一个本地运行的分阶段 3D AI 生成与渲染工作台。系统目标不是简单地把一张输入图重绘成漂亮图片，而是建立一条可控链路：先生成或载入真实 3D 白模，再由用户在浏览器中旋转、缩放并固定视角，随后由 Blender 按该固定视角输出几何参考通道，最后让图像模型只参考这些来自真实 3D 模型的通道完成最终渲染。

核心原则：最终 AI 渲染阶段只允许使用 Blender 根据实际 3D 模型和锁定相机渲染出的 rgb + edge 通道，禁止直接引用传给 Hunyuan3D 的原始输入图。

当前实现已经从“一键流水线”升级为可交互工作台：3D 生成、3D 预览定帧、白模通道渲染、AI 渲染、Agent 自动调优与报告输出都被拆成独立阶段。用户能明确看到每一步的状态，也可以在白模生成后手动选择视角，而不是被固定视角限制。

系统目标

- ☐ 用 Hunyuan3D 2.1 生成或载入高质量白模，保留真实 mesh 作为最终参考源。
- ☐ 在前端提供真实 GLB/OBJ 交互预览，支持拖转、缩放、平移、重置和预设视角。
- ☐ 将用户当前预览框中的相机位置精确传给 Blender，而不是使用默认视角。
- ☐ AI 渲染阶段使用模型渲染通道作为几何约束，降低造型漂移。
- ☐ 通过 Agent 记录每轮提示词、seed、reference channels、分数与决策，减少手工试错。

二、完整技术流程

- 用户在 Web 工作台选择来源：提示词生成 3D、图片生成 3D、已有模型路径或程序化测试模型。
- 若使用提示词生成 3D，系统先通过 HiDream-01-Image 生成参考图；若使用图片生成 3D，则直接进入 Hunyuan3D 预处理。
- Hunyuan3D 2.1 shape pipeline 生成 GLB 白模，并输出 metadata、预处理图和 mesh sanity 检查结果。
- 前端使用本地 vendored Three.js、GLTFLoader、OBJLoader 和 OrbitControls 显示真实模型，用户可拖转、滚轮缩放、平移并固定视角。

5. 前端锁定或渲染白模时发送标准化 CameraState: azimuth_deg、elevation_deg、distance_scale、ortho_scale、target、position、viewport_aspect 和 coordinate_space。
6. 后端将 Three.js 的 Y-up 相机状态转换为 Blender 的 Z-up 坐标，并调用 Blender 单视角 view_locked 渲染 rgb、depth、edge、normal、mask。
7. AI 渲染阶段读取 render_manifest，只从 view_locked 的 rgb + edge 通道构造参考，不接收原始输入图。
8. Agent 根据提示词变体、seed、guidance、steps 与本地 CV 评分选择候选，输出 final.png、white_vs_final.png 和 agent_report.json。

阶段接口

阶段	接口 / 命令	输入	输出
3D 生成	POST /api/stage/3d	source_mode、prompt、reference_image、model_path	model_path、model_url、metadata、mesh_sanity
白模渲染	POST /api/stage/white-render	model_path、camera、resolution、samples	render_manifest、white_image、channel_urls
AI 渲染	POST /api/stage/ai-render	render_manifest、prompt、negative_prompt、agent_render	final_image、comparison_image、agent_report
Agent CLI	local3dai agent-render	input-renders、output、prompt、model-key、seed	agent_trials、agent_report.json、final.png

三、3D 生成与白模质量链路

Hunyuan3D 阶段的输入质量直接决定白模质量。当前脚本按照官方思路增加了背景去除与透明主体预处理：保留原始参考图，同时输出 shape_input_preprocessed.png。该预处理图用于减少底板、背景碎片和多余几何对 shape pipeline 的干扰。



Figure 1: Hunyuan3D 输入预处理图。背景已去除并扩展为方图，主体作为 shape pipeline 的主要参考。

指标	当前结果
模型文件	white_mesh.glb
顶点数	312,693
面数	625,706
包围盒长宽高比例	3.78513
组件数量	6
最大组件面占比	0.999546
mesh sanity	pass

背景去除	<code>used_background_remover = true</code>
------	---

mesh sanity 检查关注顶点/面数、包围盒比例、组件数量、最大组件占比以及异常大底板等风险。若白模不合格，前端会把状态标成 needs_review，而不是把低质量模型伪装成成功结果继续向后推进。

四、前端 3D 预览与相机锁定

前端工作台现在是真实的 3D 交互界面，而不是静态状态页。GLB/OBJ 文件会被加载进 Three.js 场景，OrbitControls 负责拖转、缩放和平移。用户固定视角后，系统记录当前相机的 target 与 position。最新修复进一步保证：点击“渲染白模”前会重新读取当前交互框相机，所以即使用户固定后又微调了画面，Blender 也会按当前预览角度渲染。

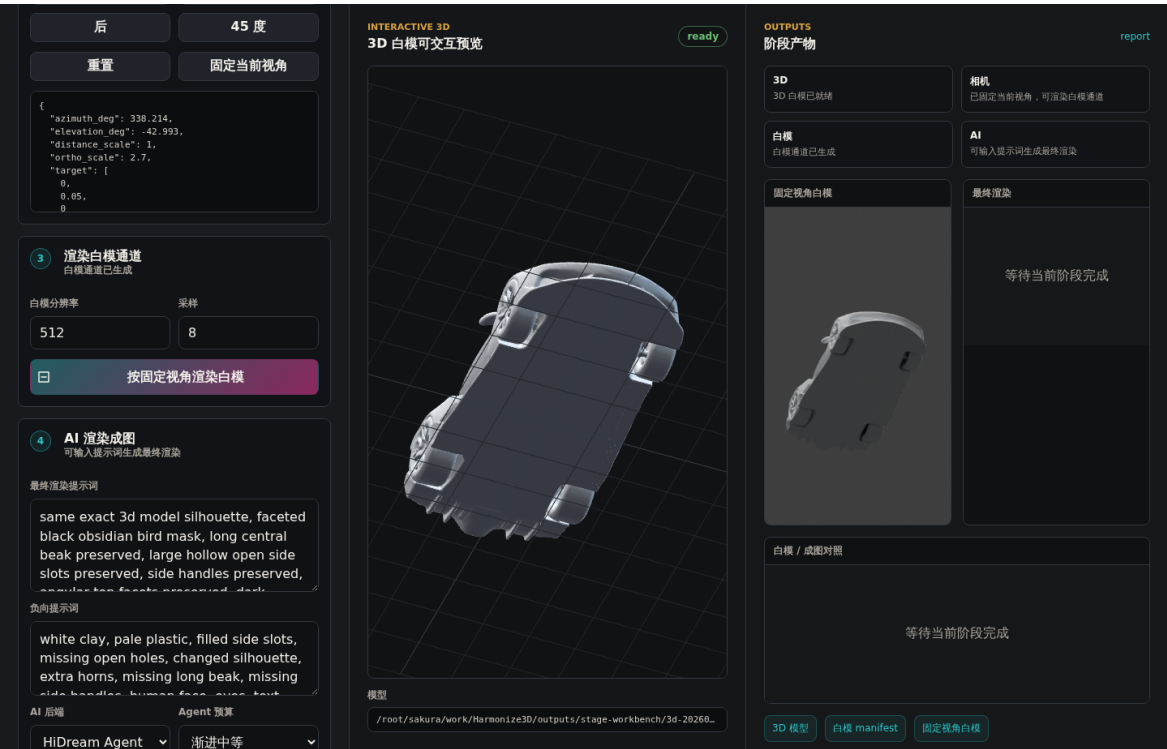


Figure 2: 真实 GLB 跑车模型拖拽后渲染白模的前端验证。右侧白模视角与中间交互预览一致。

坐标契约

glTF 规范与 Three.js 默认均为 Y-up，而 Blender 使用 Z-up。系统不再在前端强行旋转 GLB，也不修改 Three.js 全局 up 向量；而是在后端边界统一转换 CameraState。对 Three.js 相机状态，target 和 position 都按 (x, y, z) -> (x, -z, y) 转换到 Blender

空间。

JSON

```
{
  "view_id": "view_locked",
  "camera": {
    "azimuth_deg": 340.172,
    "elevation_deg": -40.89,
    "target": [0.0, -0.0, 0.05],
    "position": [-0.8206, -2.2757, -2.0447],
    "coordinate_space": "blender_z_up"
  }
}
```

五、Blender 白模通道渲染

白模渲染阶段由 Blender 脚本 `batch_render.py` 完成。固定视角模式下，脚本只输出 `view_locked` 视图，并生成 `rgb`、`depth`、`edge`、`normal`、`mask` 五类通道。AI 渲染默认只使用 `rgb` + `edge`、`depth` 和 `normal` 作为实验通道保留，因为此前结果显示它们容易放大噪点和粗糙纹理。



Figure 3: 按用户拖拽后的固定视角渲染出的白模 rgb 通道。

通道	用途	默认是否用于最终 AI
rgb	白模明暗、主体轮廓、整体视角	是
edge	轮廓线和结构边界约束	是
mask	前景区域、评分和对照图辅助	否，评估使用

depth	深度关系与空间信息	实验通道
normal	表面法线与朝向信息	实验通道

六、AI 渲染与 Agent 自动调优

AI 渲染阶段以 HiDream-01-Image full 为主。Agent v1 不替代图像模型，而是在渲染阶段自动选择提示词变体、seed、guidance、steps 和是否扩展多视图。默认策略先做单视图调优，达标后再扩展到三视图，避免一开始就用高预算反复试错。

安全边界：/api/stage/ai-render 会拒绝 reference_image、input_image、source_image、preprocessed_image 等字段。最终图像模型只能读取 render_manifest 中记录的模型渲染通道。



Figure 4: HiDream Agent 根据真实 3D 模型的 rgb + edge 通道生成的最终渲染图。

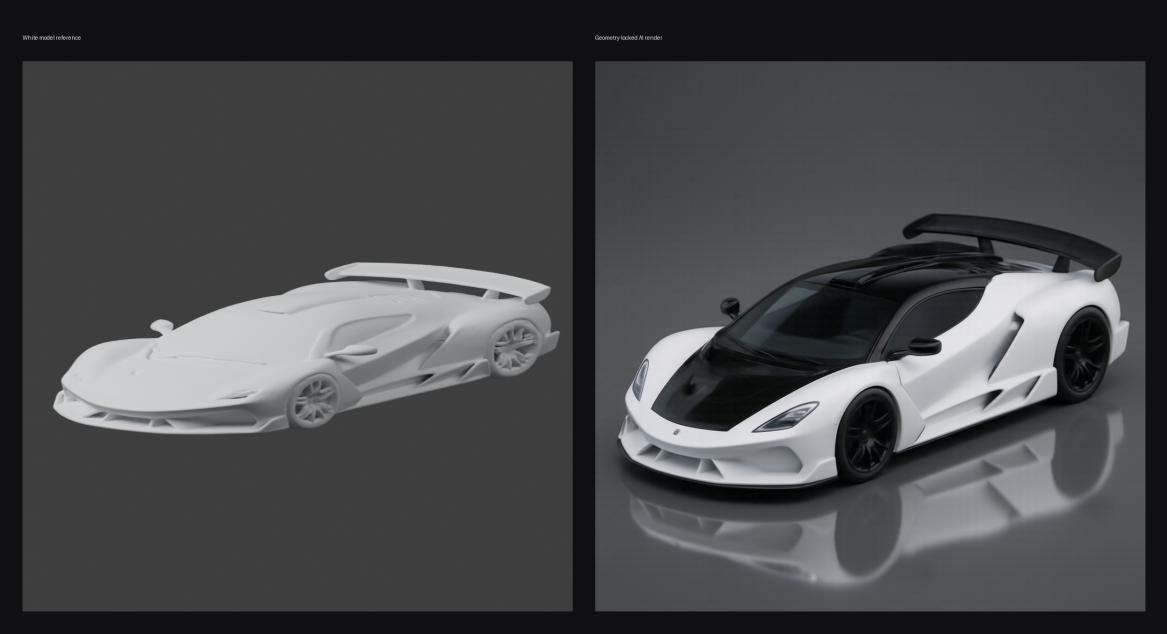


Figure 5: 白模与最终渲染对照。左侧来自 Blender 白模通道，右侧为 geometry-locked AI render。

Agent 字段	当前记录
reference_policy	model_render_channels_only
reference_channels	rgb + edge
source_model_path	white_mesh.glb
edge_f1	0.035240
mask_iou	0.420527
roughness	0.933638
background_cleanliness	0.954219
total	0.494008
status	needs_review

当前示例的 Agent 状态仍为 `needs_review`，原因是本地 CV 评分中的 edge F1 对精确轮廓非常严格。视觉结果已经可用，但算法认为局部边缘与白模仍有偏移，因此没有把它标记为 `pass`。这个设计比简单降低阈值更诚实：它能提醒用户当前结果适合查看，但仍可能需要继续调优。

七、前后端工作台效果

Web 端现在按阶段展示状态，不再让用户误以为整条流程已经完成。每个阶段都有独立按钮、禁用状态和产物链接。最终页面会同时显示 3D 预览、固定视角白模、最终渲染和白模/成图对照。

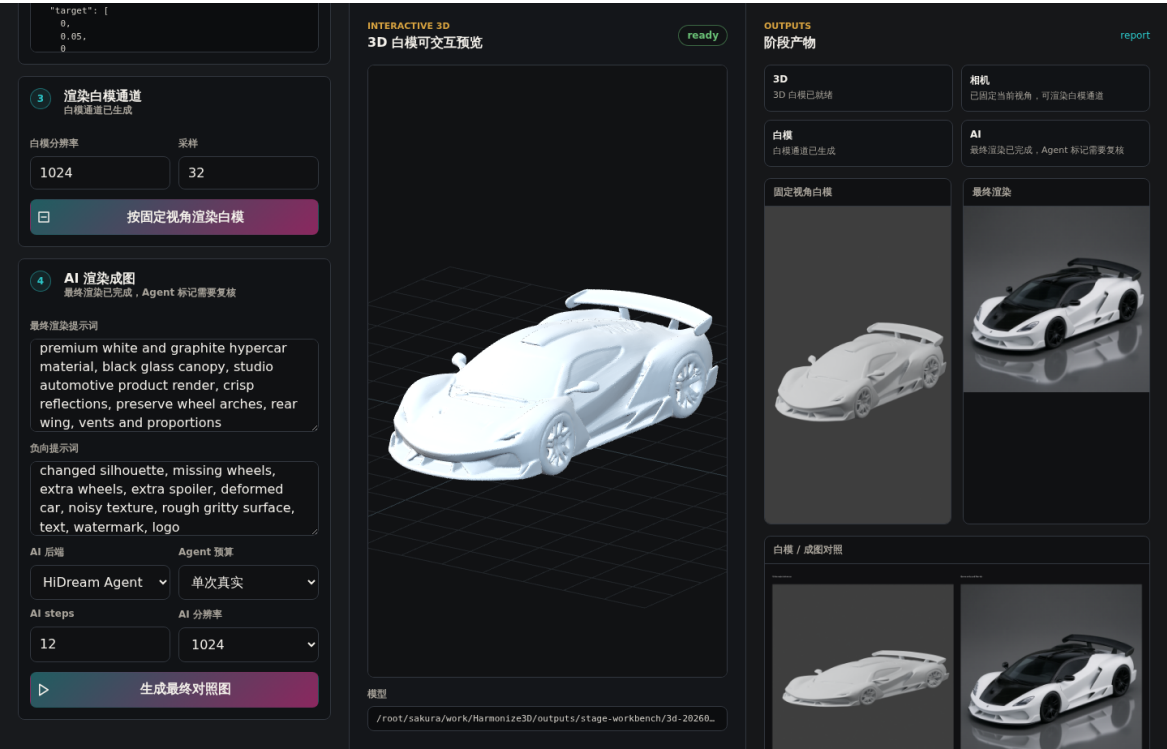


Figure 6: 完整工作前端到端验证截图：真实 GLB、固定视角白模、最终 AI 图和对照图均正常显示。

验证项	结果
单元测试	python3 -m pytest -q: 17 passed
格式检查	git diff --check: 通过
前端点击生成 3D	Playwright 通过
拖拽后白模渲染	Playwright 通过, manifest 包含拖拽后的 camera.position
最终图显示	final.png 与 white_vs_final.png 均加载成功
服务地址	http://127.0.0.1:7867

八、关键实现文件

文件	职责
web/app.js	分阶段前端工作台、Three.js 预览、CameraState 捕获、图片加载状态
src/local3dai/webapp.py	阶段式 API、Hunyuan/Blender/Agent 调度、禁止原始图进入 AI reference
src/local3dai/camera.py	Three Y-up 到 Blender Z-up 的相机状态转换
blender_scripts/batch_render.py	固定视角 view_locked 渲染 rgb/depth/edge/normal/mask
scripts/run_hunyuan_shape_white.py	Hunyuan3D shape 调用、背景去除、预处理图和 mesh sanity
src/local3dai/agent.py	Agent 自动调优、评分、候选选择、agent_report.json 输出
scripts/validate_web_workbench.mjs	Playwright 端到端验证脚本

九、结论与后续方向

Harmonize3D 当前已经形成了可运行的本地闭环：Hunyuan3D 负责 3D 白模，Three.js 负责真实交互定帧，Blender 负责几何通道，HiDream 负责最终视觉渲染，Agent 负责渐进式调优和质量记录。最新修复解决了白模渲染不跟随用户自定义视角的问题，使“所见即所渲染”成为可靠契约。

后续优化重点可以集中在两个方向：第一，提升 3D mesh 阶段的稳定性，例如更强的多视图输入、车体部件分割和 Hunyuan 参数自适应；第二，提升 AI 渲染与白模的一致性评分，例如引入更鲁棒的结构相似度、局部轮廓匹配或轻量 VLM 评估，而不是只依赖 Canny edge F1。